

Methods 4

John Dong

Problem 1

a.

The number of words on each page are as follows:

- $M_1 = 60$
- $M_2 = 69$
- $M_3 = 73$
- $M_4 = 56$
- $M_5 = 48$
- $M_6 = 43$
- $M_7 = 52$
- $M_8 = 76$
- $M_9 = 50$
- $M_{10} = 57$
- Meaning, the sampling weights would be $\frac{1331}{10} \frac{M_i}{20}$ for each word in each i^{th} cluster.

b.

$m_i = 20$, meaning we take a random sample of 20 words from each page. No page has a number of words, M_i , exceeding 20, so we don't have to adjust m_i to be smaller for any pages.

One way to randomly sample each page is to index the words on each page by counting them in order. Then, sampling 20 numbers from $\{1, 2, 3, \dots, M_i\}$.

In base R, this would look like:

```
set.seed(0)
M_i = c(60, 69, 73, 56, 48, 43, 52, 76, 50, 57)

table = matrix(nrow = 0, ncol = 20)

for (i in seq_along(M_i)) {
  row = sample(seq(M_i[i]), 20)
  table = rbind(table, row)
}

table = as.data.frame(table)
table
```

	V1	V2	V3	V4	V5	V6	V7	V8	V9	V10	V11	V12	V13	V14	V15	V16	V17	V18	V19	V20
row	14	57	4	39	1	34	23	43	60	18	33	21	49	42	46	10	7	9	15	48
row.1	37	69	34	44	33	20	35	6	10	42	38	47	64	28	57	66	23	62	40	54
row.2	25	70	39	51	42	6	24	32	14	2	45	18	22	65	38	1	72	57	40	68
row.3	23	44	6	40	39	11	17	36	13	53	25	46	56	20	52	29	48	22	41	28
row.4	44	33	45	21	31	38	17	9	39	23	19	26	30	32	29	10	1	11	27	37
row.5	15	29	24	12	39	41	28	22	33	19	26	3	11	1	42	14	36	6	20	9
row.6	37	46	49	3	22	35	7	47	44	41	27	51	20	24	16	19	36	2	1	13
row.7	66	19	17	34	31	35	46	75	16	40	9	50	28	58	24	10	15	3	37	57
row.8	32	37	39	31	49	29	12	14	38	16	15	2	1	3	9	17	20	5	36	27
row.9	44	31	53	8	36	49	41	29	21	1	20	42	46	24	22	38	16	30	50	2

where row i represents the indices of the words sampled for the i^{th} cluster.

c.

My results for how many words I knew:

- Page 45: 13

- Page 563: 12
- Page 825: 15
- Page 122: 10
- Page 341: 11
- Page 177: 13
- Page 1354: 13
- Page 553: 12
- Page 600: 15
- Page 586: 13

```
know_count = c(13, 12, 15, 10, 11, 13, 13, 12, 15, 13)
data = data.frame()

for (i in seq_along(M_i)) {
  row = data.frame(
    page = i,
    know = c(rep(1, know_count[i]), rep(0, 20 - know_count[i])),
    M_i = M_i[i]
  )
  data = rbind(data, row)
}
rownames(data) = NULL
data
```

	page	know	M_i
1	1	1	60
2	1	1	60
3	1	1	60
4	1	1	60
5	1	1	60
6	1	1	60
7	1	1	60
8	1	1	60
9	1	1	60
10	1	1	60
11	1	1	60
12	1	1	60
13	1	1	60
14	1	0	60
15	1	0	60
16	1	0	60
17	1	0	60

18	1	0	60
19	1	0	60
20	1	0	60
21	2	1	69
22	2	1	69
23	2	1	69
24	2	1	69
25	2	1	69
26	2	1	69
27	2	1	69
28	2	1	69
29	2	1	69
30	2	1	69
31	2	1	69
32	2	1	69
33	2	0	69
34	2	0	69
35	2	0	69
36	2	0	69
37	2	0	69
38	2	0	69
39	2	0	69
40	2	0	69
41	3	1	73
42	3	1	73
43	3	1	73
44	3	1	73
45	3	1	73
46	3	1	73
47	3	1	73
48	3	1	73
49	3	1	73
50	3	1	73
51	3	1	73
52	3	1	73
53	3	1	73
54	3	1	73
55	3	1	73
56	3	0	73
57	3	0	73
58	3	0	73
59	3	0	73
60	3	0	73

61	4	1	56
62	4	1	56
63	4	1	56
64	4	1	56
65	4	1	56
66	4	1	56
67	4	1	56
68	4	1	56
69	4	1	56
70	4	1	56
71	4	0	56
72	4	0	56
73	4	0	56
74	4	0	56
75	4	0	56
76	4	0	56
77	4	0	56
78	4	0	56
79	4	0	56
80	4	0	56
81	5	1	48
82	5	1	48
83	5	1	48
84	5	1	48
85	5	1	48
86	5	1	48
87	5	1	48
88	5	1	48
89	5	1	48
90	5	1	48
91	5	1	48
92	5	0	48
93	5	0	48
94	5	0	48
95	5	0	48
96	5	0	48
97	5	0	48
98	5	0	48
99	5	0	48
100	5	0	48
101	6	1	43
102	6	1	43
103	6	1	43

104	6	1	43
105	6	1	43
106	6	1	43
107	6	1	43
108	6	1	43
109	6	1	43
110	6	1	43
111	6	1	43
112	6	1	43
113	6	1	43
114	6	0	43
115	6	0	43
116	6	0	43
117	6	0	43
118	6	0	43
119	6	0	43
120	6	0	43
121	7	1	52
122	7	1	52
123	7	1	52
124	7	1	52
125	7	1	52
126	7	1	52
127	7	1	52
128	7	1	52
129	7	1	52
130	7	1	52
131	7	1	52
132	7	1	52
133	7	1	52
134	7	0	52
135	7	0	52
136	7	0	52
137	7	0	52
138	7	0	52
139	7	0	52
140	7	0	52
141	8	1	76
142	8	1	76
143	8	1	76
144	8	1	76
145	8	1	76
146	8	1	76

147	8	1	76
148	8	1	76
149	8	1	76
150	8	1	76
151	8	1	76
152	8	1	76
153	8	0	76
154	8	0	76
155	8	0	76
156	8	0	76
157	8	0	76
158	8	0	76
159	8	0	76
160	8	0	76
161	9	1	50
162	9	1	50
163	9	1	50
164	9	1	50
165	9	1	50
166	9	1	50
167	9	1	50
168	9	1	50
169	9	1	50
170	9	1	50
171	9	1	50
172	9	1	50
173	9	1	50
174	9	1	50
175	9	1	50
176	9	0	50
177	9	0	50
178	9	0	50
179	9	0	50
180	9	0	50
181	10	1	57
182	10	1	57
183	10	1	57
184	10	1	57
185	10	1	57
186	10	1	57
187	10	1	57
188	10	1	57
189	10	1	57

```

190  10    1  57
191  10    1  57
192  10    1  57
193  10    1  57
194  10    0  57
195  10    0  57
196  10    0  57
197  10    0  57
198  10    0  57
199  10    0  57
200  10    0  57

```

```

data$wt = (1331/10)*(data$M_i/20) #from part a
data$N = 1331

des1 = svydesign(~page + know, weights = ~wt, fpc = ~ N + M_i, data = data)

```

By using two-stage cluster samplings, we have to pass multiple parameters so that R knows what to look at. In order to tell R to know that we are first sampling pages, we pass the “page” column as our largest cluster id, which says which sampled words belong to which cluster/page, and N (total number of pages) as our first fpc. Then, for our next id, we pass “words”, as our second id parameter so that R knows to know we are sampling words within each page. Then, we need to specify our second fpc, M_i, the total number of words on each page.

As for weights, we know that conceptually, two-stage cluster sampling consists of a random sample of n clusters, and within each of those clusters, there is some number of records sampled m_i among the cluster total M_i . This gives us the sampling probability $\pi_{ij} = \frac{n}{N} \frac{m_i}{M_i}$, or a weight of $\frac{N}{n} \frac{M_i}{m_i}$, subbing 1331 pages for N , 10 pages for n , respective words-per-page totals for M_i , and 20 for m_i .

d.

```

total_know = svytotal(~know, des1)

total_know

```

```

      total      SE
know 49440 3937.1

```

Estimated to know a total of **49,440** words with a standard error of 3937.1.

e.

```
know_pp = svytotal(~know, des1)/ 1331
```

```
know_pp
```

```
      total      SE  
know 37.145 3937.1
```

```
SE(total_know)/ 1331
```

```
      know  
know 2.957966
```

Estimated to know **37** words per page with a standard error of 2.96.

f.

```
perc_know = svymean(~know, des1) * 100
```

```
perc_know
```

```
      mean      SE  
know 63.604 0.028
```

Estimated to know **63.6%** of words in the dictionary with a standard error of 2.8%.

2.

a.

Jacoby and Handlin “selected 26 from a list of 1285 scholarly journals” and “examined all articles” for the selected journals. One record for this survey is represented by an article, and a scholarly journal is a compilation of several articles. Thus, by collecting articles by how they are naturally grouped in journals makes the journals natural **clusters**, and all articles in each journal is examined, making it akin to **one-stage cluster sampling**.

b.

```
# used image of csv from Canvas to feed into Duke GPT for data.frame() arguments
journal = data.frame(
  journal_id = 1:26,
  numemp = c(
    17, 1, 3, 3, 23, 3, 18, 1, 0, 5, 0, 0, 13,
    0, 0, 0, 0, 46, 3, 5, 2, 0, 0, 1, 4, 0
  ),
  prob = c(
    0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0,
    0, 0, 0, 0, 0, 0, 0, 0, 0, 2, 0, 0
  ),
  nonprob = c(
    17, 1, 3, 2, 19, 3, 16, 1, 0, 4, 0, 0, 13,
    0, 0, 0, 0, 46, 3, 5, 1, 0, 0, 1, 2, 0
  )
)

journal$N = 1285
journal$n = 26
journal$wt = (journal$N / journal$n)

des2 = svydesign(~journal_id, weights = ~wt, fpc = ~N, data = journal) #only one id and fpc p

non_prob_prop <- svyratio(numerator = ~nonprob, denominator = ~numemp, design = des2)

non_prob_prop
```

```
Ratio estimator: svyratio.survey.design2(numerator = ~nonprob, denominator = ~numemp,
  design = des2)
Ratios=
      numemp
nonprob 0.9256757
SEs=
      numemp
nonprob 0.03398672
```

We get an estimated proportion of 0.926 for the proportion of social/behavioral journal studies that use nonprobability sampling with a standard error of 0.034.

c.

This statement is misleading because it makes some false assumptions about the population being sampled from. We are told the sample comes from a list of 1285 scholarly journals containing articles in the social and behavioral sciences. Therefore, the proportion we calculated describes the proportion of social and behavioral sciences that are published in a scholarly journal and use nonprobability sampling.

However, the statement goes on to use the estimated proportion to claim that this means that nonprobability is common enough for courts to have no problem with it being used. While it is likely for the majority of articles in social/behavioral science journals to use nonprobability sampling, this is a specific subdomain of articles. Additionally, social and behavioral sciences fundamentally rely more on voluntary participation, which would mean that these journals can disproportionately be more reliant on nonprobability sampling. We also don't know if these articles are credible enough to be presented in court. A scholarly study can still be released in a journal, but then be deemed outdated or inaccurate later on. A stronger argument could be made if the articles were sampled from a population of articles deemed suitable for litigation by some outside metric, and if the sample wasn't restricted to one field of study.

3.

a.

Recall:

$$Var(\hat{t}_y) = \frac{N^2(1 - \frac{n}{N})S_{(t)}^2}{n}$$

$$\text{where } S_{(t)}^2 = \sum_{i=1}^N \frac{(t_{yi} - \bar{T}_y)^2}{N - 1}$$

```
hypo = read.csv("~/sta322_methods4/data/HypotheticalClusterPopulation.csv")  
  
# T bar  
hypo |> summarize(T_bar = sum(y)/3)
```

```
T_bar  
1 1581
```

```
# t_1
hypo |> filter(cluster == 1) |> summarize(sum = sum(y))
```

```
sum
1 29
```

```
# t_2
hypo |> filter(cluster == 2) |> summarize(sum = sum(y))
```

```
sum
1 648
```

```
# t_3
hypo |> filter(cluster == 3) |> summarize(sum = sum(y))
```

```
sum
1 4066
```

$$S_{(t)}^2 = \sum_{i=1}^3 \frac{(t_{yi} - 1581)^2}{3 - 1}$$

$$S_{(t)}^2 = \frac{(29 - 1581)^2}{2} + \frac{(648 - 1581)^2}{2} + \frac{(4066 - 1581)^2}{2}$$

```
((29-1581)^2)/2 + ((648-1581)^2)/2 + ((4066-1581)^2)/2
```

```
[1] 4727209
```

$$S_{(t)}^2 = 4,727,209$$

$$Var(\hat{t}_y) = \frac{N^2(1 - \frac{n}{N})S_{(t)}^2}{n}$$

$$Var(\hat{t}_y) = 3^2(1 - \frac{1}{3})4727209^2$$

```
(9*2/3)*(4727209)
```

```
[1] 28363254
```

$$\text{Var}(\hat{t}_y) = 28,363,254$$

b.

$$\text{Var}(\hat{t}_y) = \frac{N^2(1 - \frac{n}{N})S^2}{n}$$

$$\text{Var}(\hat{t}_y) = \frac{30^2(1 - \frac{10}{30})S^2}{10}$$

$$S^2 = \frac{1}{N-1} \sum_{i=1}^N (y_i - \bar{Y})^2$$

```
hypo = hypo |> mutate(Y_bar = sum(y)/30) |> mutate(diff = (y-Y_bar)^2)  
hypo
```

	cluster	y	Y_bar	diff
1	1	3	158.1	24056.01
2	1	2	158.1	24367.21
3	1	4	158.1	23746.81
4	1	3	158.1	24056.01
5	1	3	158.1	24056.01
6	1	2	158.1	24367.21
7	1	4	158.1	23746.81
8	1	5	158.1	23439.61
9	1	2	158.1	24367.21
10	1	1	158.1	24680.41
11	2	60	158.1	9623.61
12	2	62	158.1	9235.21
13	2	64	158.1	8854.81
14	2	65	158.1	8667.61
15	2	62	158.1	9235.21
16	2	67	158.1	8299.21
17	2	71	158.1	7586.41

18	2	72	158.1	7413.21
19	2	63	158.1	9044.01
20	2	62	158.1	9235.21
21	3	404	158.1	60466.81
22	3	406	158.1	61454.41
23	3	403	158.1	59976.01
24	3	410	158.1	63453.61
25	3	410	158.1	63453.61
26	3	409	158.1	62950.81
27	3	412	158.1	64465.21
28	3	402	158.1	59487.21
29	3	404	158.1	60466.81
30	3	406	158.1	61454.41

```
hypo |> summarize(S_2 = sum(diff)/29)
```

```
      S_2
1 32610.58
```

$$S^2 = \frac{1}{29} \sum_{i=1}^{30} (y_i - 158.1)^2$$

$$S^2 = 32610.58$$

$$Var(\hat{t}_y) = \frac{30^2(1 - \frac{10}{30})(32610.58)}{10}$$

```
(30^2)*(2/3)*(32610.58)/10
```

```
[1] 1956635
```

$$Var(\hat{t}_y) = 1,956,635$$

c.

The variance for the H-T estimator in the cluster sample is much greater because in our data, we can observe that each cluster each have a range of values that then vary drastically from the other clusters. This means that, when we get our estimator only using $n = 1$, i.e. only estimating the total from one cluster, we miss out on a majority of the range of y .

This is different from the SRS model because, with SRS, the random process gives us a decent chance of collecting data from each cluster, while not guaranteed but much better than missing out on most clusters entirely.

Then, we can see how this is represented by looking at what is different between the two S^2 formulas. In the cluster sample, we see that S^2 has a much greater magnitude when the cluster totals deviate far from the average cluster totals, which is going to inevitably happen for this data because our clusters have egregious differences between their respective ranges of y .

d.

For each cluster, the finite population variance

$$S_h^2 = \frac{1}{N_h - 1} \sum_{i=1}^{N_h} (y_{hi} - \bar{Y}_h)^2, \text{ where}$$

$$\bar{Y}_h = \frac{1}{N_h} \sum_{i=1}^{N_h} y_{hi}$$

So, first we compute our finite population variances:

```
fin_pop_var = hypo |>
  group_by(cluster) |>
  mutate(strat_avg = mean(y)) |>
  summarize(strat_var = sum(((y - mean(y))^2)/9)) |>
  select(cluster, strat_var)
```

```
fin_pop_var
```

```
# A tibble: 3 x 2
  cluster strat_var
  <int>     <dbl>
1     1      1.43
2     2     16.2
3     3     11.8
```

Using formula, $Var(\hat{t}_y) = \sum_{h=1}^H N_h^2(1 - \frac{n_h}{N_h}) \frac{S_h^2}{n_h}$,

$$Var(\hat{t}_y) = \sum_{h=1}^3 10^2(1 - \frac{3}{10}) \frac{S_h^2}{3} = 10^2(1 - \frac{3}{10}) \frac{1.4\bar{3}}{3} + 10^2(1 - \frac{3}{10}) \frac{16.1\bar{7}}{3} + 10^2(1 - \frac{3}{10}) \frac{11.8\bar{2}}{3}$$

```
fin_pop_var |> summarize(var = sum((70/3)*(strat_var)))
```

```
# A tibble: 1 x 1
  var
<dbl>
1 687.
```

$$Var(\hat{t}_y) = 686.\bar{7}$$

e.

Our population consists of these three equally sized groups, called clusters, that are densely concentrated at 3 points. The first cluster's y values are centered around ~3, the second around ~65, and the third around ~407. Each cluster's y values are spaced far apart from each other, meaning that the total population has a wide variety of y values. However, if we want to make more consistent estimates of the total value for y in the population, we can use the fact that each of the groupings of y are consistently centered around a value. If we take a small sample from each group, we are more likely to get similar totals from each group to then estimate our population total. This is different than if we were to neglect the groupings, because then we would be assuming that the groups are similar when they are actually very different.

4.

When comparing the correct theoretical formula,

$$Var(\hat{t}_y) = \frac{N^2(1 - \frac{n}{N})S_{(t)}^2}{n};$$

$$\text{where } S_{(t)}^2 = \sum_{i=1}^N \frac{(t_{yi} - \bar{T}_y)^2}{N - 1}$$

or–

$$\text{Var}(\hat{t}_y) = \frac{3^2(1 - \frac{1}{3})S_{(t)}^2}{1};$$

$$\text{where } S_{(t)}^2 = \sum_{i=1}^3 \frac{(\sum_{j=1}^{10} y_{ij} - \frac{1}{3} \sum_{i,j \in S} y_{ij})^2}{3 - 1}$$

to the friend's estimated variance

$$\hat{\text{Var}}(\hat{t}_y) = \frac{30^2(1 - \frac{10}{30})s^2}{10}$$

we know that the friend will consistently have an underestimate of the variance. This is because their s^2 's estimates are calculated using only the data collected from sampling one cluster. However, as we already established, the clusters are drastically different and have largely varying totals between them. Therefore, the friend is missing any variance between clusters, which was the main contributor to our theoretical variance.

We can confirm this by checking the different possibilities for the friend's variance estimates using our within-cluster variances from before:

- Cluster 1: $\hat{\text{Var}}(\hat{t}_y) = \frac{30^2(1 - \frac{10}{30})1.43}{10} = 86$
- Cluster 2: $\hat{\text{Var}}(\hat{t}_y) = \frac{30^2(1 - \frac{10}{30})16.17}{10} = 970$
- Cluster 3: $\hat{\text{Var}}(\hat{t}_y) = \frac{30^2(1 - \frac{10}{30})11.82}{10} = 709.3,$

which are all egregiously under our theoretical variance.

5.

We have to check the expected value for the new estimator:

$$\begin{aligned} \mathbb{E}[\hat{t}_y^*] &= \mathbb{E}[\sum_{i \in S, i \neq i^*} w_i y_i + w y_{i^*}] \\ &= \sum_{i \in \mathcal{U}, i \neq i^*} \mathbb{E}[w_i y_i I_i] + \mathbb{E}[w y_{i^*} I_{i^*}] \\ &= \sum_{i \in \mathcal{U}} w_i y_i \mathbb{E}[I_i] - w_{i^*} y_{i^*} \mathbb{E}[I_{i^*}] + w y_{i^*} \mathbb{E}[I_{i^*}] \\ &= \sum_{i \in \mathcal{U}} w_i y_i p_i - w_{i^*} y_{i^*} p_{i^*} + w y_{i^*} p_{i^*} \end{aligned}$$

$$= \sum_{i \in \mathcal{U}} \frac{1}{p_i} y_i p_i - \frac{1}{p_{i^*}} y_{i^*} p_{i^*} + w y_{i^*} p_{i^*}$$

$$= \sum_{i \in \mathcal{U}} y_i + y_{i^*} (w p_{i^*} - 1)$$

$$\mathbb{E}[\hat{t}_y^*] = t_y + y_{i^*} (w p_{i^*} - 1) = t_y + y_{i^*} \left(\frac{w}{w_{i^*}} - 1 \right)$$

Therefore, the estimator is biased, because the expected value can only come out to t_y if the weights w and w_{i^*} are equal, which they're not. So, the bias is

$$Bias = \mathbb{E}[\hat{t}_y^*] - t_y = y_{i^*} \left(\frac{w}{w_{i^*}} - 1 \right)$$

And we know that the bias is negative because $w < w_{i^*}$.